ROME RTL Generation and Coverage Report

I extended the original ROME notebook with a hierarchical 8-bit adder and integrated the course iVerilog+Covered flow. Two rounds of targeted test additions increased instance-level coverage to 100%, with all 31 final tests passing.

1. **Design and generation**

   The design is half_adder → full_adder → adder4 → adder8. Each full_adder uses two half_adders; adder4 uses four full_adders; adder8 uses two adder4 instances linked by carry. Interfaces use a, b, cin, sum and cout, with half_adder using carry and no cin. Bus widths are 4 and 8 bits at the corresponding levels.

   I used OpenAI gpt-4o with API defaults and max_iterations=10 in the original ROME RTL loop. No explicit temperature or seed was set. Fixed self-checking baseline testbenches passed 4, 8, 512 and 8 tests respectively. Keys were loaded from the Colab secret ROME.

2. **Mux verification and repair**

   All six original mux levels passed their supplied tests. mux16_1 initially failed because a child output drove output reg out. ROME changed the port to the net output out in iteration 1, with the testbench fixed, and simulation passed. Original compiler feedback and repair logs are retained.

   The PPA optimizer ran 10 iterations per mux. Best transistor proxies for mux2/4/8/16/32/64 were 12/36/60/180/372/756; cell counts were 1/3/5/15/31/63. Timing was N/A. These raw Yosys proxies are not measured physical area or power; hierarchical cell counts are not a normalized physical comparison.

3. **Coverage integration and results**

   After passing ROME simulation, the top testbench dumped a VCD. Python compiled all dependencies, ran vvp, checked success, cleaned VCD comments, scored Covered at adder8tb.dut and saved reports per iteration. Actual uncovered reports and the current TB were sent to the LLM; its JSON vectors were appended using the existing check task. RTL hashes and text-preservation assertions kept RTL and existing checks fixed.

Iteration	Tests	Line	Toggle ↑	Toggle ↓	Logic
Baseline	8	40/40	141/189	137/189	98/136
Refine 1	25	40/40	185/189	185/189	128/136
Refine 2	31	40/40	189/189	189/189	136/136

   Round 1 added 17 tests for missing toggles, carry paths and XOR/AND combinations. Round 2 added 6 tests for bit-4/7 carry generation and missing high-half XOR combinations. RTL stayed

fixed in both rounds; all tests passed, so no further RTL repair was needed. Module-merged coverage hid instance gaps, so the table uses instance totals. Full measured coverage is not a proof of correctness for all inputs.

4. **Final generation prompts**

These exact design prompts were used with ROME's system instructions and fixed testbench context. Complete prompts and LLM responses are included in the submission.

Adder8

Generate a hierarchical 8-bit unsigned ripple-carry adder in Verilog-2001.

Exact interface:

```
module adder8(
    input wire [7:0] a,
    input wire [7:0] b,
    input wire cin,
    output wire [7:0] sum,
    output wire cout
);
```

Available previously generated submodule:

```
module adder4(
    input wire [3:0] a,
    input wire [3:0] b,
    input wire cin,
    output wire [3:0] sum,
    output wire cout
);
```

Requirements:

- Compute a + b + cin, with the 9-bit result {cout, sum}.
- Instantiate exactly two adder4 instances.
- The low instance handles a[3:0] and b[3:0].
- Connect cin to the low instance.
- Connect the low instance carry output to the high instance cin.
- The high instance handles a[7:4] and b[7:4].
- Connect the high instance carry output to cout.
- Connect both partial sums to the corresponding sum slices.
- Use combinational logic only.
- Do not implement the addition using the + operator.

- Output only adder8.
- Do not repeat any submodule definitions.
- Do not generate or modify the testbench.

Full_adder

Generate a hierarchical full adder in Verilog-2001.
Exact interface:

```
module full_adder(
    input wire a,
    input wire b,
    input wire cin,
    output wire sum,
    output wire cout
);
```

Available previously generated submodule:

```
module half_adder(
    input wire a,
    input wire b,
    output wire sum,
    output wire carry
);
```

Requirements:
- Compute a + b + cin, with result {cout, sum}.
- Instantiate exactly two half_adder instances.
- The first half_adder adds a and b.
- The second half_adder adds the first sum and cin.
- Combine the two carry outputs with an OR gate.
- Use combinational logic only.
- Output only full_adder.
- Do not repeat the half_adder definition.
- Do not generate or modify the testbench.

5. Final verification

```
cd adder/final
iverilog -g2001 -s adder8tb -o final.vvp half_adder.v full_adder.v adder4.v adder8.v tb_final.v
vvp final.vvp
```

covered score -t adder8 -i adder8tb.dut -v half_adder.v -v full_adder.v -v adder4.v -v adder8.v -vcd adder8.vcd -o final.cdd
covered report -m ltc -d s -i -o final_summary.rpt final.cdd
Saved simulation logs confirm “adder8: all 31 tests passed!” Covered reports confirm 100% for all four metrics. Recorded tools: Icarus Verilog 12.0 and Covered 0.7.10. Baseline/refined TBs, CDDs, VCDs, prompts, RTL repair logs and coverage_history.json are retained. Results were checked against archived evidence; packaging did not rerun simulation.